

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב - מואץ
67130 – מבוא למדעי המחשב - רגיל
67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 5.2.2012
משך הבחינה: שלוש שעות

מועד א' - סמסטר א' - תשע"ב – 2011-2012
פרופ' דני לישצ'ינסקי, פרופ' נעם ניסן

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (34 נק'), אך הציון המקסימלי הוא 100.
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת **במחברת נפרדת**. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות** או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
- בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

שאלה 1 (34 נקודות)

נתונה המחלקה הבאה לייצוג רשימה מקושרת:

```
public class List {
    public String key;
    public String value;
    public List next;
    public List(String key,String value,List next){
        this.key=key; this.value=value; this.next=next;
    }
}
```

כמו כן, נתון ה-Interface הבא המתאר מבנה נתונים המגדיר מיפוי בין מפתחות (keys) לערכים (values), כאשר לכל מפתח יכול להיות ממופה ערך אחד או יותר. בנוסף, צמד מסוים של מפתח-ערך יכול להופיע יותר מפעם אחת:

```
public interface MultiMap{
```

```
    /**מוסיף ערך למפתח מסוים*/
```

```
    public void put(String key, String value);
```

```
    /** מסיר את כל הערכים עבור מפתח מסוים, אם קיים כזה. */
```

```
    public void remove(String key);
```

```
    /**null מחזיר רשימה המכילה את כל הערכים המשויכים למפתח מסוים, אם קיים כזה. אחרת, מחזיר null.*/
```

```
    public List get(String key);
```

```
}
```

א. (30 נקודות=9 נקודות לכל שיטה+3 נקודות עבור כתיבה תקינה של המחלקה)

ממשו את המחלקה MultiMapWithList המממשת את ה-interface הנתון, באמצעות המחלקה הנתונה List.

ב. (4 נקודות) חשבו את סיבוכיות כל אחת מהשיטות אשר כתבתם בסעיף א'.

הערה: שים לב כי בשאלה זו שדות המחלקה List הוגדרו כ-public וניתן לגשת אליהם ישירות ללא שימוש בשיטות המיועדות לכך.

שאלה 2 (34 נקודות)

א. מספרי קטלן (סה"כ 17 נקודות)

מספר הסדרות ה"מאוזנות" של התווים '(' ו-')' עם n תווים מכל סוג נקרא מספר קטלן (Catalan) ומסומן ב C_n . מספרי קטלן מוגדרים באמצעות נוסאת הנסיגה הבאה:

$$C_0 = 1$$
$$C_n = \sum_{k=1}^n C_{k-1} \times C_{n-k}$$

1. (8 נקודות) עליכם לממש שיטה רקורסיבית שעבור כל מספר נתון, n , תחשב את מספר קטלן המתאים, C_n . אינכם נדרשים לפתרון יעיל בסעיף זה.

```
public static int catalan(int n)
```

סמנו על גבי הקוד מהו מקרה הבסיס.

2. (9 נקודות) ממשו את אותה שיטה, אך הפעם באופן יעיל וללא רקורסיה (יעילות כאן היא בזמן $O(n^2)$). ניתן להקצות ולהשתמש במבני נתונים אשר גודלם ומספרם תלוי בגודל הקלט.

ב. Selection Sort (מוכר גם כ: Max sort) (סה"כ 17 נקודות)

בסעיף זה אין להשתמש בלולאות כלל!

1. (8 נקודות) ממשו באופן רקורסיבי את השיטה הסטטית הבאה:

```
private static int argmax(Comparable[] a, int n)
```

אשר מחזירה את האינדקס של האבר הגדול ביותר בתחום $a[0]..a[n-1]$.

2. (9 נקודות) ממשו באופן רקורסיבי את השיטה הסטטית הבאה:

```
public static void sort(Comparable[] data)
```

השיטה מקבלת מערך של אובייקטים שממשים את הממשק Comparable, ממיינת אותו באמצעות אלגוריתם selection sort ובסיום ריצתה המערך ממויין בסדר עולה. בסעיף זה אין להשתמש בלולאות מכל סוג, אך ניתן להשתמש במתודות עזר.

הערה: Comparable אינו גנרי כאן, ומוגדר כ:

```
interface Comparable {
```

```
    public int compareTo(Comparable other);
```

מחזיר מספר שלילי אם האובייקט הנוכחי קטן מ- $other$, מספר חיובי אם הוא גדול יותר,

ו-0 אם הם שווים מבחינת סדר ההשוואה.

```
}
```

שאלה 3 (34 נקודות)

נגדיר Interface שמתאר פעולה (פונקציה) המקבלת זוג מספרים שלמים בתחום $0, 1, \dots, N-1$ ומחזירה מספר שלם בתחום זה:

```
public interface Op {  
    public int getN();  
    public int Op(int x, int y)  
}
```

א. (10 נקודות) כתבו מחלקה בשם Add, המממשת את ה-Interface הזה, המקבלת בקונסטרקטור שלה N שלם ומייצרת אובייקט המתאר חיבור מודולו N. כלומר, את הפונקציה המחשבת $(x+y)\%N$.

ב. (10 נקודות) כתבו מחלקה בשם Inverse, המממשת את ה-Interface הזה, המקבלת בקונסטרקטור שלה פעולה מטיפוס Op ומחזירה את הפעולה ה"הפוכה", כלומר: אם פעולת הקלט הייתה $f()$ אזי פעולת הפלט $g(x,y)$ תוגדר כערך z עבורו $f(z,y)=x$. (לדוגמא, $\text{new Inverse}(\text{new Add}(N))$ תחזיר את פעולת החיסור מודולו N. באם אין כזה z אזי ערך g מוגדר כ-0; אם יש יותר מ-z יחיד אזי יש להחזיר את הקטן ביותר.

ג. (9 נקודות) כתבו שיטה סטטית:

```
public static Boolean isCommutative(Op f)
```

המחזירה ערך TRUE באם הפעולה מקיימת את חוק החילוף (כלומר לכל x,y מתקיים $(f(x,y)=f(y,x))$.

ד. (5 נקודות) נתחו את זמן הריצה של כל המתודות שכתבתן לעיל.

שאלה 4 (34 נקודות)

שאלה זו עוסקת במידול של בית-ספר ומשתמשת בשלש מחלקות שאלו הממשקים שלהם:

```
class Student {
    public Student(String name) // מייצר סטודנט חדש ללא הגדרת מגמה
    public String getName() // מחזיר את שם הסטודנט
    public Major getMajor() // מחזיר את המגמה בה לומד סטודנט זה, אם קיימת כזו. אחרת
    null מחזיר
    public void setMajor(Major newMajor) // קובע (אם הייתה כבר -- מחליף) את
    המגמה של סטודנט זה
    public boolean likes(String subject) // האם הסטודנט אוהב את הנושא הזה
}

// מגמת לימוד בבית הספר
class Major {
    public Major(String subject) // מייצר מגמה חדשה (ריקה מתלמידים) בנושא הנתון
    public Student[] getStudents() // מחזיר מערך באורך מספר הסטודנטים במגמה המכיל את
    הסטודנטים במגמה (ולא העתקים שלהם)
    public void addStudent(Student s) // מוסיף סטודנט למגמה. פעולה זו כוללת עדכון
    אצל הסטודנט ואצל המגמה הקודמת (אם הייתה) של הסטודנט הרישום
}

class School {
    public School() // מייצר בית ספר (ריק) חדש
    public void addMajor(Major major) // מוסיף מגמת לימוד (קיימת) על כל תלמידיה לבית
    הספר

    public void addMajor(String subject) // מייצר מגמה חדשה בנושא הנתון ומעביר אליה
    את כל הסטודנטים מכל המגמות הקיימות כבר בבית הספר אשר אוהבים את הנושא
}
```

א. (18 נקודות=9 נקודות לכל מחלקה)

ממשו את המחלקות Major ו-School (אין לממש את Student). ניתן להקצות מערכים התלויים בגודל הקלט אך אסור להשתמש בספריית ה-Collections של Java. ניתן להניח חסמים קבועים מראש על מספרי התלמידים והמגמות. בסעיף זה אין לשנות את הממשקים הנתונים. ניתן להוסיף שדות ושיטות פרטיות כנדרש.

ב. (9 נקודות) כתבו שיטה:

```
public static void String[] recommendedMajors(School school, Student student)
```

אשר מחזירה את רשימת הנושאים שהסטודנט הנתון אוהב אשר עבורם קיימת מגמת לימוד בבית-הספר הנתון. בסעיף זה ניתן להוסיף שיטות (גם פומביות) למחלקות שמימשותם בסעיף הקודם. יש לממש כל תוספת כזו ולציין בפירוש היכן מקומה.

ג. (7 נקודות) נתחו את זמן הריצה של כל מתודה (וקונסטרקטור) שכתבתם. ניתן להניח כי סיבוכיות זמן הריצה של כל השיטות במחלקה Student הוא $O(1)$.